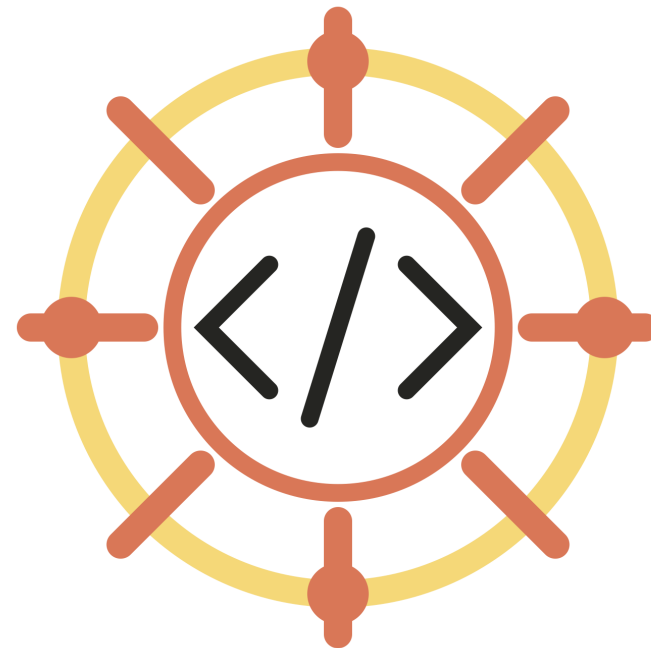


Anthropic

2 万亿 IPO 的背后

编码模型与 AGI

- Claude 为什么强?
- 编码模型如何持续进步?
- 为什么 AGI 需要编码能力?



两万亿的想象，来自工作能力的扩张

资本数字只是入口：开发工具 → 数字工作 → 智能研发。

\$2T

IPO 目标估值

01

写代码

构建、修复与维护软件

02

完成数字工作

组合数据、工具与业务系统

03

参与智能研发

进入下一代模型的研发过程

市场押注的，不只是一款工具，而是更大范围的任务委托。

四次转折：理解代码、选择行动、持续执行

终端入口把模型能力变成了可反复运行的工作方式。

2024.06



3.5 Sonnet

更强的代码修改
速度与能力兼顾

2025.02



3.7 + Code

可调思考 + 终端
查、改、测进入同一任务

2025.05



Claude 4

工具与思考交错
长任务与 Code 正式可用

2026.09



Sonnet 5.5

减少冗余步骤
日常工程执行更高效

Claude 的强，体现在四种工程判断

不是输出更多代码，而是更常把任务推进到可用结果。

工程判断	可靠执行	缺失时的代价
找准问题	沿调用链识别原因	反复修改表面症状
控制改动	保持约束，限定修改范围	修好一处，破坏另一处
选择证据	运行能区分假设的检查	搜索很多，信息增量很少
完成交付	出错后恢复，核对真实结果	留下半成品，让人接管
判断与行动配合，决定任务是否需要人反复接管。		

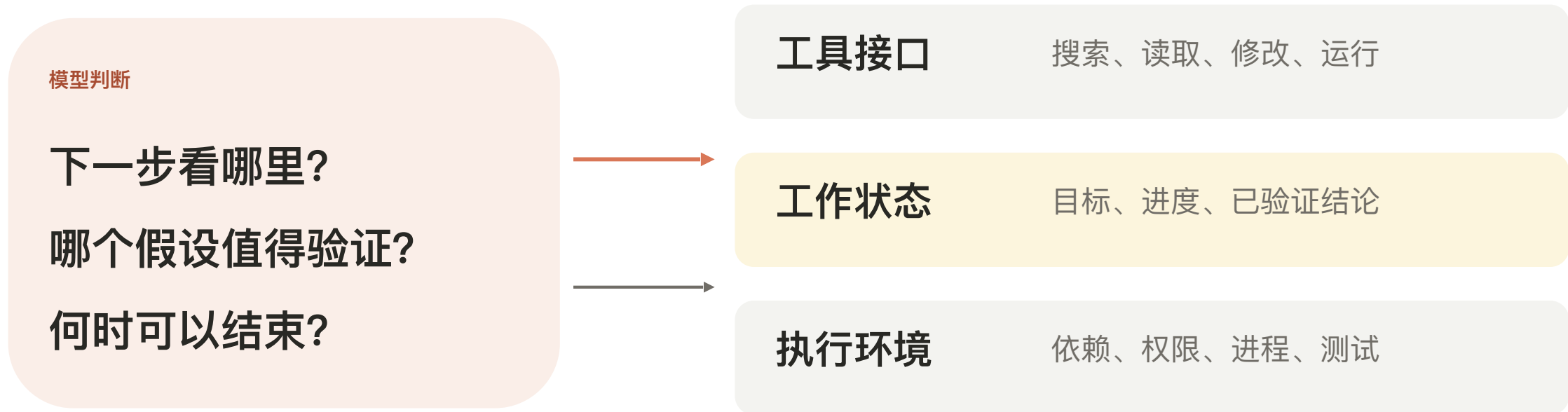
连接耗尽只是症状，修复要看懂生命周期

任务取消后，Redis 连接没有归还，最终耗尽连接池。



Claude Code 把模型接入一套工作系统

同一个模型，工具接口、上下文与环境不同，交付能力也会不同。



Harness： 让判断变成真实行动，再把环境反馈带回模型的执行系统。

大仓库的理解，靠按需建立高价值上下文

窗口容量决定能装多少；检索策略决定现在该看什么。

一次塞进整座仓库



大量无关实现挤占工作记忆

围绕当前问题逐层探索

项目规则 + 测试入口

相关函数 + 调用关系

最近失败 + 已验证结论

回显有多清楚，下一步判断就有多准确

Agent 通过工具感知工程环境；错误信息也是行动材料。

低信息量回显

```
edit_file()  
→ operation failed
```

不知道哪里失败
继续猜参数、重复尝试

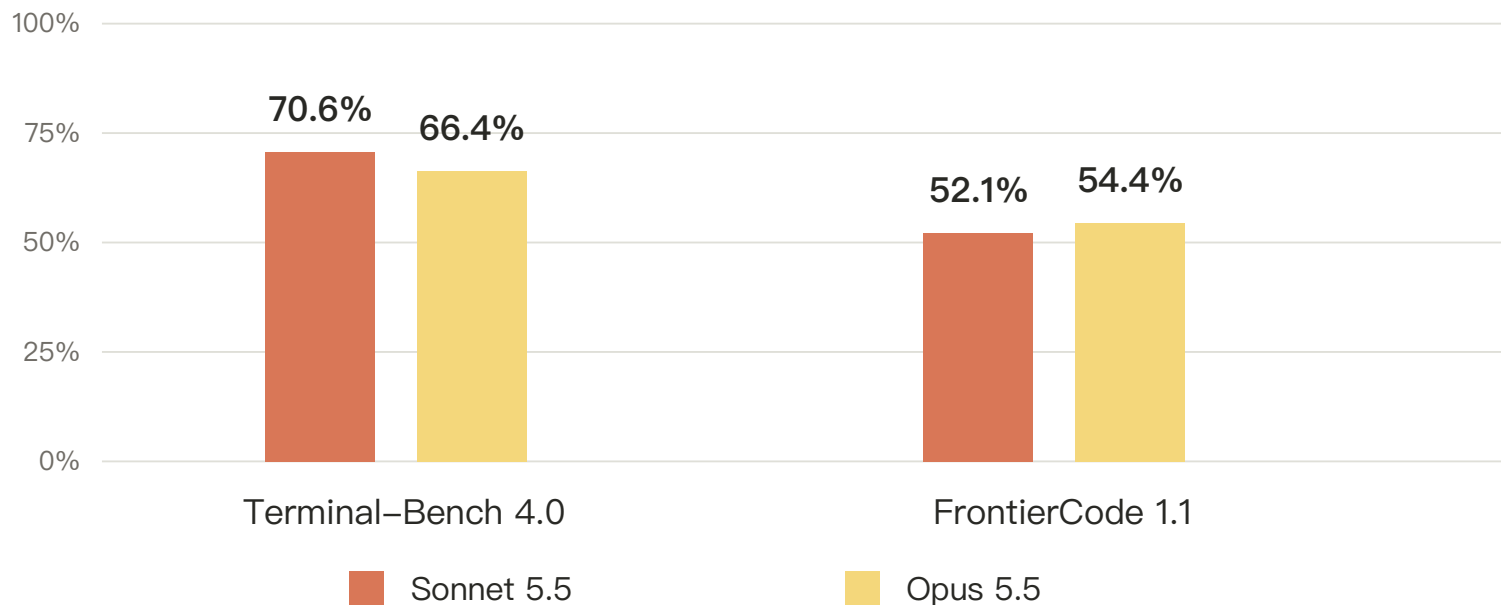
可定位的回显

```
edit_file(path, expected_text)  
→ 未匹配位置 + 当前片段
```

把失败变成新证据
缩小范围，精确修正

Sonnet 的优势：把能力用在合适的任务上

终端执行效率，与开放难题中的持续判断，是不同的优势。



任务匹配

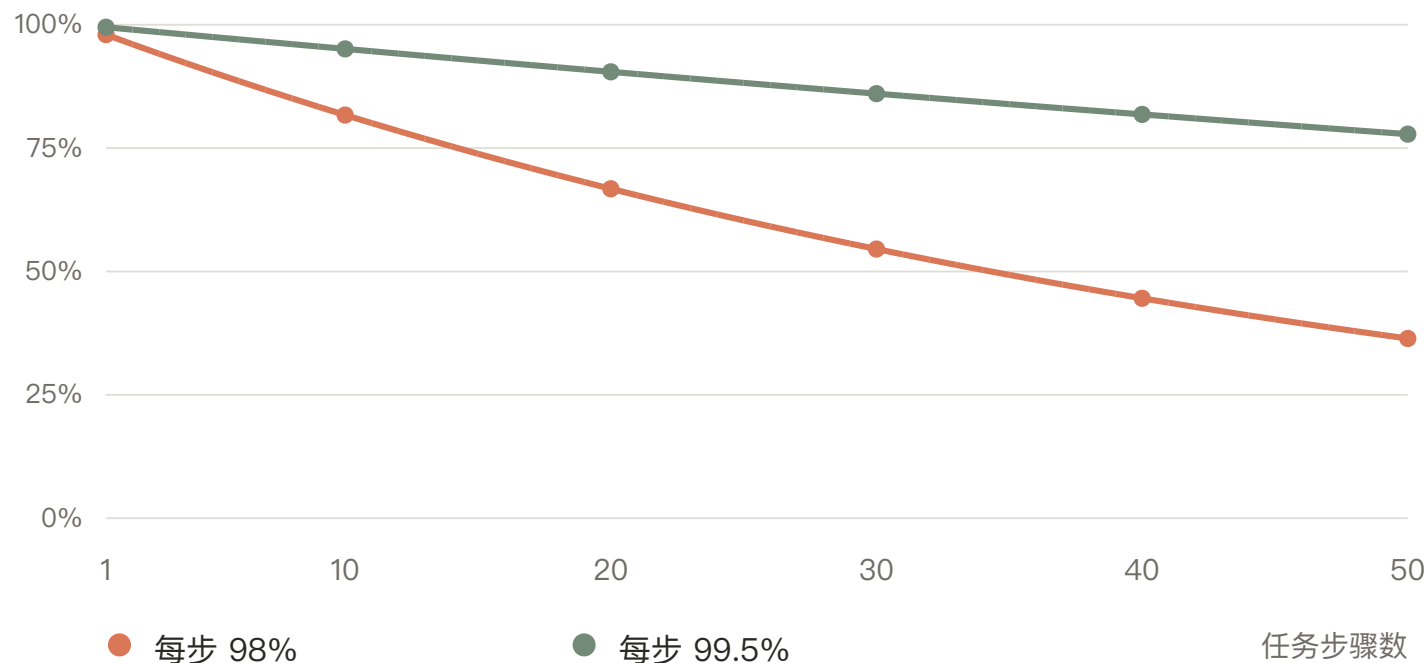
日常实现
速度与协作

开放难题
持续判断

更少步骤是效率线索；一个榜单，不能独自解释全部工程体验。

小错误会在长任务里累积放大

简化示意：50 个独立步骤，每步可靠性恒定，失败不可恢复。



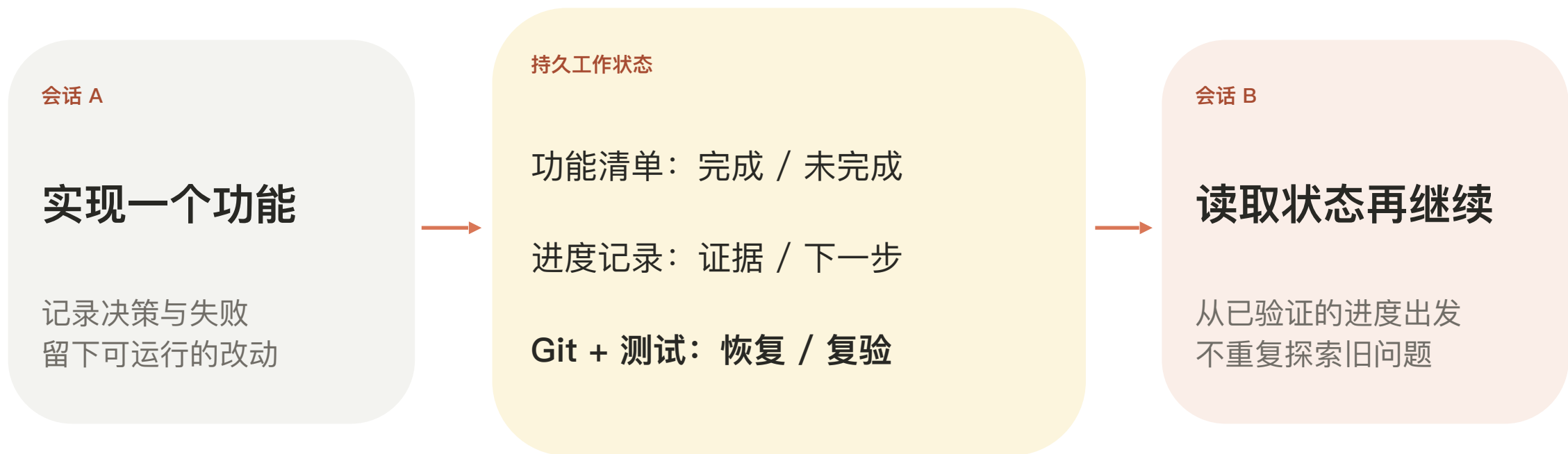
36% → 78%

真实体验还取决于

错误后能否恢复
需要人接管几次
结果能否放心合并

长任务的连续性，靠可恢复的工作状态

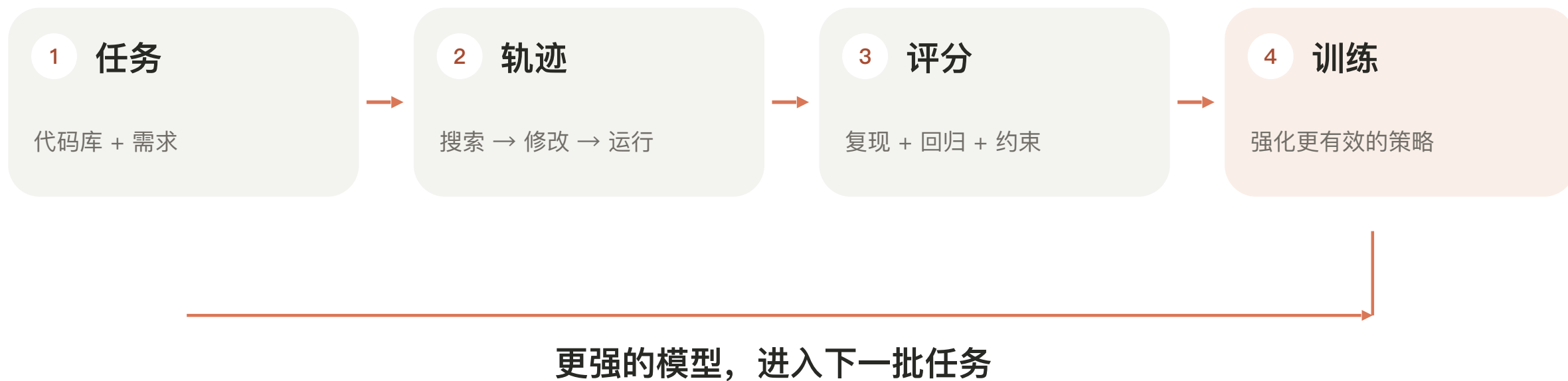
下一次会话接续的，不只是聊天历史，而是可复验的进度。



文件记忆让系统能够接班；它不等于模型权重已经持续学习。

代码环境，提供可规模化的训练反馈

任务、执行环境、评分器，让解题过程成为可重复的训练材料。



任务内纠错，与模型训练，是两种不同的进步

一个改变当前方案，一个改变之后处理任务的能力。

执行阶段

根据报错修正方案

读取失败 → 改写补丁 → 再跑测试

同一任务里，工作方案变得更好。

训练阶段

根据轨迹更新模型

收集尝试 → 环境评分 → 训练更新

下一批任务里，解题策略变得更好。

评测在追问：Agent 完成了哪一层工作？

修复补丁、处理复杂仓库、操作终端、达到可合并质量。

1

SWE-bench Verified

真实 Issue 的补丁
能否通过测试？

2

SWE-bench Pro

复杂、长程的仓库任务
能否推进到底？

3

Terminal-Bench

终端操作之后
系统是否符合要求？

4

FrontierCode

改动质量
是否达到可合并标准？

成绩同时受到模型、工具系统、预算、运行环境与评分规则影响。

配置能启动，不代表访问控制做对了

Nginx 示例：只允许持有有效客户端证书的请求访问。

语法检查

`nginx -t`

配置是否合法？

正向请求

有效证书 → 接受

该放行的能否放行？

反向请求

无 / 错证书 → 拒绝

该拦截的能否拦截？

代码世界的优势，是大量可运行的裁判；裁判质量决定反馈质量。

工程任务里，廉价反馈能减少无效推演

先运行能区分假设的检查，把不确定性变成证据。

只有脑内推演

猜环境 → 猜依赖 → 猜失败原因 → 一次大改

用检查获取证据

小检查



环境回显



修正假设



小改动

快速反馈与深度思考互补：关键是下一步能否减少不确定性。

Codex 的追赶，也在结合模型与执行系统

编码不是边缘垂类，而是前沿模型的重要产品与训练方向。



2025.04.16

Codex CLI

本地仓库与终端

同机修改、运行、检查



2025.05.16

云端 Codex

独立沙箱与并行任务

读取代码、跑测试、提交结果

codex-1 已披露真实编码任务强化学习：学习修改、测试与继续修复。

很多办公室工作，本来就是未写出来的程序

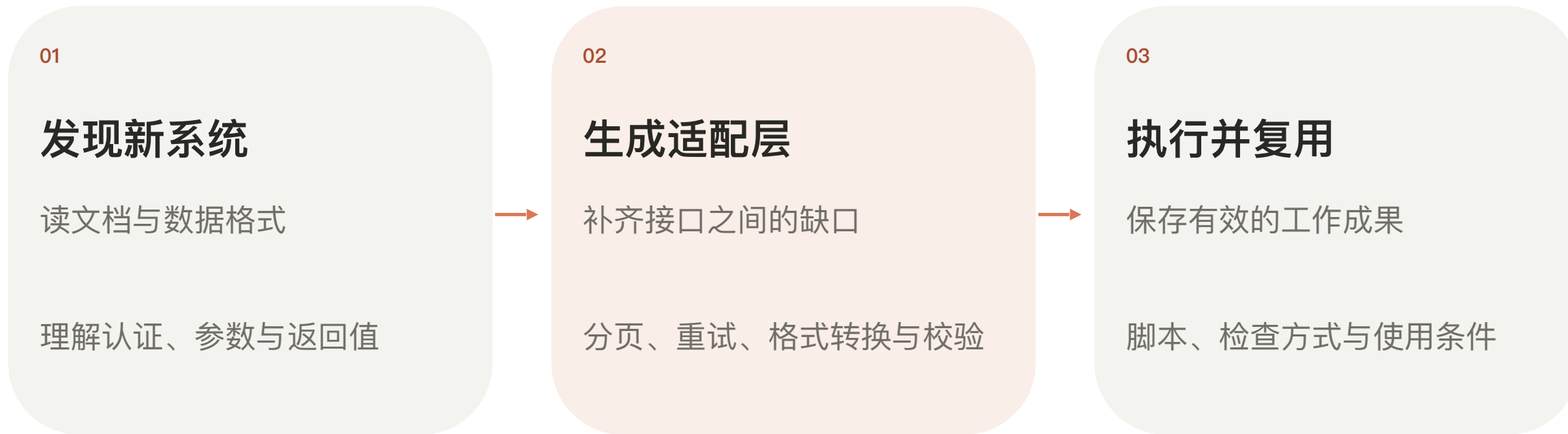
例：核对本月报销，找出重复扣款与不一致记录。



条件、循环、数据与异常处理，过去由员工在不同系统间亲自执行。

会写代码，Agent 就能按需制造工具

预定义工具覆盖已知任务；生成适配层，扩展新的行动组合。



CLI、MCP、Skills，接成一条实际工作流

例：订单同步失败——获取线索、复现故障、修复并交付。

MCP

连接业务

日志、工单、业务数据

CLI

执行工程

搜索、运行、修改、测试

Skills

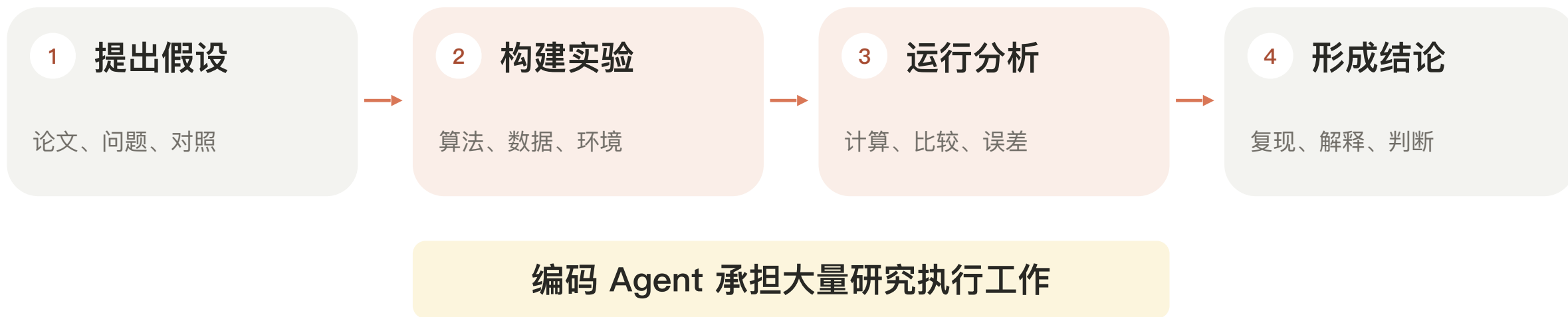
复用经验

诊断顺序、脚本、交付标准

同一任务里：连到系统 → 操作环境 → 复用组织经验

科学研究也需要把想法变成可执行实验

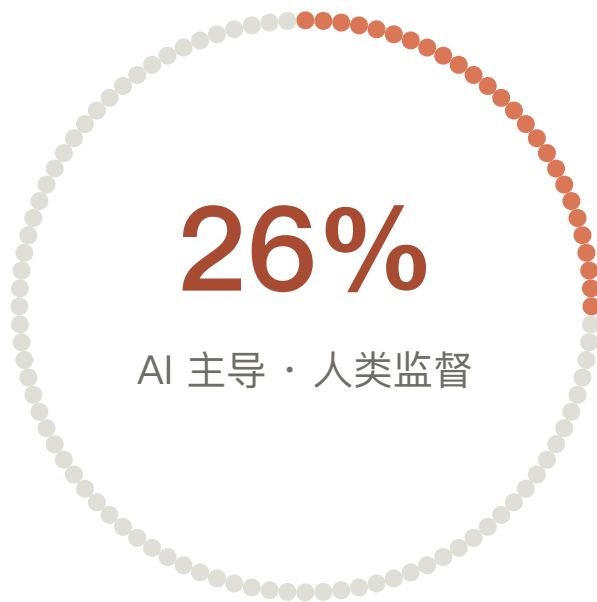
编码降低研究执行成本，科学判断决定结果有没有意义。



实验运行成功 $\neq$ 假设成立；真实增益仍要靠对照、统计与复现。

Claude 已经进入下一代 Claude 的研发

研发自动化指数：任务从 AI 协作，向 AI 主导推进。



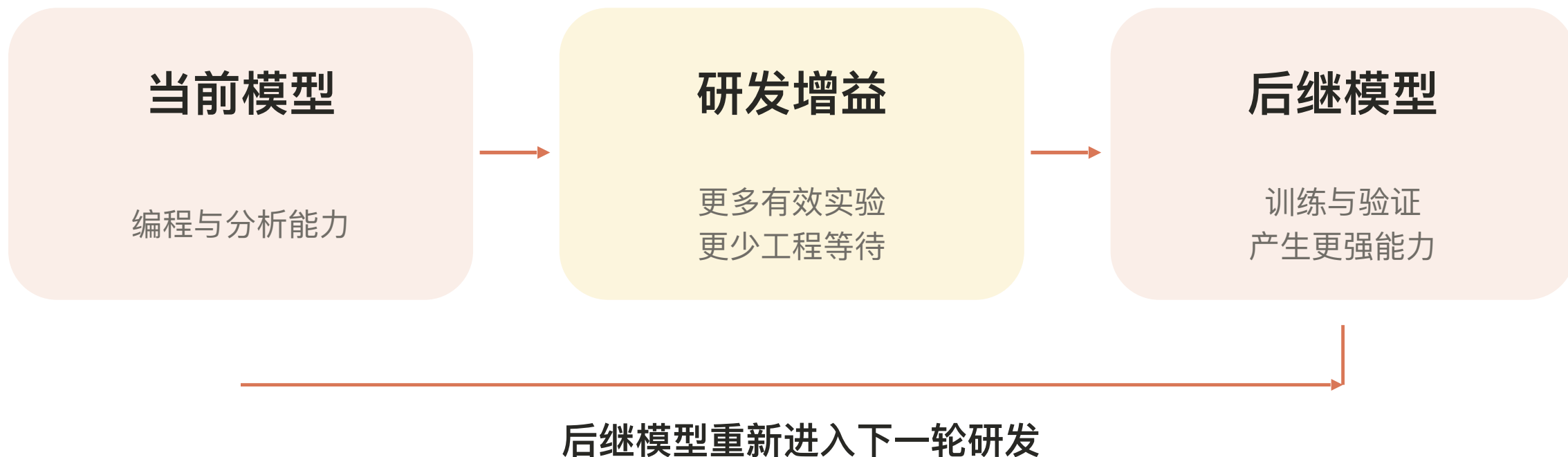
>90%

至少达到 AI 协作

完全自主：所测任务类别尚未报告
指标描述参与程度，不等于原创突破占比

RSI：研究工具开始参与改进研究工具

递归发生在代际之间：当前模型参与研发，后继模型再进入研发。



Software 3.0: 自然语言成为新的编程入口

Prompt、模型权重与传统代码，共同构成新的工作系统。

3.0

自然语言

定义目标、约束与工作方式

2.0

模型权重

提供识别、推理与行动判断

1.0

显式代码

访问数据、执行规则与连接系统

软件既可以是固定产品，也可以成为针对一次任务、按需生成的能力。

执行越来越便宜，业务判断越来越重要

不是每个人都要成为程序员，但每个人都要更会定义与验收任务。

岗位	过去的执行重心	新的能力重心
运营 / 市场	取数、整理、制作报表	定义指标、提出假设、识别异常
财务 / HR	核对材料、跨系统录入	建立规则、核对证据、处理例外
产品	写需求、画静态原型	用可运行成果验证业务假设
研发	逐行实现与重复调试	架构、复杂问题、验收与责任

瓶颈正在从“有没有人做”，转向“什么值得做”

不只给旧流程加 AI，而是重新审视排期、交接与等待。

旧问题

怎样增加执行资源？

更多人手

更多软件席位

更多排期与交接

新问题

哪些过去做不起的事值得做？

更有价值的问题

可执行的组织知识

可检验的业务结果

程序员是第一批用户，智能研发是更大的应用

从工程交付，到通用行动，再到参与制造下一代智能。

01

Claude 的工程优势

判断 × 工具 × 工作状态

更少接管与返工

02

编码模型的成功机制

任务 × 环境 × 反馈

可规模化地练习行动

03

通往 AGI 的路径

制造工具 × 研究执行

进入后继智能的研发

如果执行不再稀缺，你会怎样重新组织工作？